

Celebel Week 2 Assignment using tables provided in word document

Reference: https://celebaltech.sharepoint.com/:w:/s/Celebal-LMS/IQDQ_Co-E08_RZ03eKkixGR-AQ09RkM5df72wSBZjaL9LYc?rttime=x6G8ibPE3kg

Section A — SQL Basics (SELECT, Constraints, Primary Keys)

Q1. SELECT * FROM customers;

	customer_id	first_name	last_name	email	city	state
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra
	108	Divya	Nair	divya.n@email.com	Kochi	Kerala
*	NULL	NULL	NULL	NULL	NULL	NULL

Q2. SELECT first_name, last_name, city FROM customers;

	first_name	last_name	city
▶	Aarav	Sharma	Mumbai
	Priya	Patel	Ahmedabad
	Rohan	Gupta	Delhi
	Sneha	Reddy	Hyderabad
	Vikram	Singh	Jaipur
	Ananya	Iyer	Chennai
	Karan	Mehta	Pune
	Divya	Nair	Kochi

Q3. SELECT DISTINCT category FROM products;

	category
▶	Clothing
	Electronics
	Home

Q4. **Table** **Primary Key**

- customers customer_id
- products product_id
- orders order_id
- order_items item_id

Explanation:

A Primary Key:

- Uniquely identifies each record in a table.
- Cannot contain duplicate values.

- Cannot contain NULL values.

Example: customer_id = 101

It must refer to exactly one customer.

If duplicate or NULL primary keys were allowed, the database would not be able to uniquely identify records, causing data integrity issues.

Q5. Column definition: email VARCHAR(100) UNIQUE NOT NULL

Constraints Applied:

- NOT NULL (Email cannot be empty.)
- UNIQUE (No two customers can have the same email address.)

Example:

```
INSERT INTO customers
VALUES (
109,
'Rahul',
'Kumar',
'aarav.s@email.com',
'Bangalore',
'Karnataka',
'2024-09-01',
FALSE
);
```

Since aarav.s@email.com already belongs to customer 101, MySQL will generate an error similar to:

```
22 09:51:38 INSERT INTO customers VALUES (109, 'Rahul', 'Kumar', 'aarav.s@email.com', 'Bangalore', 'Karnataka', '2024-09-01', FALSE) Error Code: 1062. Duplicate entry 'aarav.s@email.com' for key 'customer_email' 0.016 sec
```

The row will not be inserted.

Q6. INSERT INTO products VALUES (209, 'Test Product', 'Electronics', 'TestBrand', -50, 100);

Result: Insert fails.

```
23 09:52:31 INSERT INTO products VALUES (209, 'Test Product', 'Electronics', 'TestBrand', -50, 100) Error Code: 3819. Check constraint 'products_chk_1' is violated. 0.000 sec
```

Constraint:

CHECK (unit_price > 0)

The price is negative, so MySQL rejects the row with a CHECK constraint error.

Section B — Filtering & Optimization (WHERE, Indexes)

Q7. SELECT * FROM orders WHERE status = 'Delivered';

	order_id	customer_id	order_date	status	total_amount
▶	1001	101	2024-08-01	Delivered	4498.00
	1002	102	2024-08-03	Delivered	799.00
	1004	101	2024-08-10	Delivered	3499.00
	1006	105	2024-08-15	Delivered	5898.00
	1008	103	2024-08-20	Delivered	899.00

Q8. SELECT * FROM products WHERE category = 'Electronics' AND unit_price > 2000;

	product_id	product_name	category	brand	unit_price	stock_qty
▶	203	Smart Watch	Electronics	Noise	2999.00	150
	205	Bluetooth Speaker	Electronics	JBL	3499.00	200
*	NULL	NULL	NULL	NULL	NULL	NULL

Q9. SELECT * FROM customers WHERE state = 'Maharashtra' AND join_date BETWEEN '2024-01-01' AND '2024-12-31';

	customer_id	first_name	last_name	email	city	state	join_date	is_prem
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q10. SELECT * FROM orders WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25' AND status <> 'Cancelled';

	order_id	customer_id	order_date	status	total_amount
▶	1004	101	2024-08-10	Delivered	3499.00
	1006	105	2024-08-15	Delivered	5898.00
	1007	106	2024-08-18	Pending	1299.00
	1008	103	2024-08-20	Delivered	899.00
	1009	107	2024-08-25	Shipped	6098.00
*	NULL	NULL	NULL	NULL	NULL

Q11. idx_orders_date is an index on order_date.

Benefit: Speeds up searching and filtering orders by date because MySQL can locate matching rows quickly instead of scanning the entire table.

Example: SELECT * FROM orders WHERE order_date = '2024-08-15';

	order_id	customer_id	order_date	status	total_amount
▶	1006	105	2024-08-15	Delivered	5898.00
*	NULL	NULL	NULL	NULL	NULL

Q12. SELECT * FROM customers WHERE YEAR(join_date) = 2024;

	customer_id	first_name	last_name	email	city	state	join_date	is_prem
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1

Will this use an index? **No.**

Applying YEAR() to the column prevents efficient index usage.

Index-friendly query:

SELECT * FROM customers WHERE join_date BETWEEN '2024-01-01' AND '2024-12-31';

	customer_id	first_name	last_name	email	city	state	join_date	is
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1

This allows MySQL to use an index on join_date efficiently.

Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. SELECT COUNT(*) AS total_orders FROM orders;

	total_orders
▶	10

Q14. SELECT SUM(total_amount) AS total_revenue FROM orders WHERE status = 'Delivered';

	total_revenue
▶	17191.00

Q15. SELECT category, AVG(unit_price) AS avg_price FROM products GROUP BY category;

	category	avg_price
▶	Clothing	2699.000000
	Electronics	2224.000000
	Home	949.000000

Q16. SELECT status, COUNT(*) AS order_count, SUM(total_amount) AS total_revenue
FROM orders GROUP BY status ORDER BY total_revenue DESC;

	status	order_count	total_revenue
▶	Delivered	6	17191.00
	Shipped	2	13596.00
	Cancelled	1	2999.00
	Pending	1	1299.00

Q17. SELECT category, MAX(unit_price) AS max_price, MIN(unit_price) AS min_price
FROM products GROUP BY category;

	category	max_price	min_price
▶	Clothing	4599.00	799.00
	Electronics	3499.00	899.00
	Home	1299.00	599.00

Q18. SELECT category, AVG(unit_price) AS avg_price
FROM products GROUP BY category HAVING AVG(unit_price) > 2000;

	category	avg_price
▶	Clothing	2699.000000
	Electronics	2224.000000

Section D — Joins & Relationships

Q19. SELECT o.order_id, o.order_date, c.first_name, c.last_name, o.total_amount
FROM orders o INNER JOIN customers c ON o.customer_id = c.customer_id;

	order_id	order_date	first_name	last_name	total_amount
▶	1001	2024-08-01	Aarav	Sharma	4498.00
	1004	2024-08-10	Aarav	Sharma	3499.00
	1002	2024-08-03	Priya	Patel	799.00
	1003	2024-08-05	Rohan	Gupta	7498.00
	1008	2024-08-20	Rohan	Gupta	899.00
	1005	2024-08-12	Sneha	Reddy	2999.00
	1006	2024-08-15	Vikram	Singh	5898.00

Q20. SELECT c.customer_id, c.first_name, c.last_name, o.order_id, o.order_date
FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id;

	customer_id	first_name	last_name	order_id	order_date
▶	101	Aarav	Sharma	1001	2024-08-01
	101	Aarav	Sharma	1004	2024-08-10
	102	Priya	Patel	1002	2024-08-03
	103	Rohan	Gupta	1003	2024-08-05
	103	Rohan	Gupta	1008	2024-08-20
	104	Sneha	Reddy	1005	2024-08-12
	105	Vikram	Singh	1006	2024-08-15

Q21. SELECT o.order_id, p.product_name, oi.quantity, oi.unit_price, oi.discount_pct
FROM orders o JOIN order_items oi ON o.order_id = oi.order_id JOIN products p
ON oi.product_id = p.product_id;

	order_id	product_name	quantity	unit_price	discount_pct
▶	1001	Wireless Earbuds	2	1499.00	0.00
	1006	Wireless Earbuds	1	1499.00	10.00
	1002	Cotton T-Shirt	1	799.00	0.00
	1003	Smart Watch	1	2999.00	0.00
	1005	Smart Watch	1	2999.00	0.00
	1003	Running Shoes	1	4599.00	5.00
	1006	Running Shoes	1	4599.00	5.00
	1004	Bluetooth Speaker	1	3499.00	0.00
	1009	Bluetooth Sneaker	1	3499.00	0.00

Q22. LEFT JOIN: Returns all rows from the left table and matching rows from the right table.

Ex: SELECT * FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id;

	customer_id	first_name	last_name	email	city	state	join_date	is
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0

Shows all customers, even those without orders.

RIGHT JOIN: Returns all rows from the right table and matching rows from the left table.

Ex: `SELECT * FROM customers c RIGHT JOIN orders o ON c.customer_id = o.customer_id;`

	customer_id	first_name	last_name	email	city	state	join_date	is
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0

Shows all orders, even if a matching customer is missing.

FULL OUTER JOIN: Returns all rows from both tables.

Used when you want all customers and all orders, matched where possible.

Q23. Foreign Key Relationships

Parent Table	Child Table	Foreign Key
customers	orders	customer_id
orders	order_items	order_id
products	order_items	product_id

Ex: `INSERT INTO orders VALUES (1011, 999, '2024-08-30', 'Pending', 1000);`

42 10:09:47 INSERT INTO orders VALUES (1011, 999, '2024-08-30', 'Pendi... Error Code: 1452. Cannot add or update a child row: a foreign k... 0.015 sec

MySQL will reject the insertion with a Foreign Key Constraint Error because customer_id = 999 does not exist in the customers table. This maintains referential integrity.

Section E-Advanced Concepts (CASE, ACID, Transactions)

Q24. SELECT product_name, unit_price,
CASE
WHEN unit_price < 1000 THEN 'Budget'
WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
ELSE 'Premium'
END AS price_tier
FROM products;

	product_name	unit_price	price_tier
▶	Wireless Earbuds	1499.00	Mid-Range
	Cotton T-Shirt	799.00	Budget
	Smart Watch	2999.00	Mid-Range
	Running Shoes	4599.00	Premium
	Bluetooth Speaker	3499.00	Premium
	Bedsheet Set	1299.00	Mid-Range
	Laptop Stand	899.00	Budget
	Cushion Covers (Set)	599.00	Budget

Q25. SELECT SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS Delivered,
SUM(CASE WHEN status <> 'Delivered' THEN 1 ELSE 0 END) AS Not_Delivered
FROM orders;

	Delivered	Not_Delivered
▶	6	4

Q26. ACID Properties

A - Atomicity

Transaction happens completely or not at all.

Example: In a bank transfer, money should be deducted and credited together.

C - Consistency

Database remains valid before and after a transaction.

Example: Account balance cannot become negative if rules prohibit it.

I - Isolation

Multiple transactions do not interfere with each other.

Example: Two users withdrawing money simultaneously should not see inconsistent balances.

D - Durability

Once committed, data is permanently saved.

Example: After a successful transfer, data remains even if the system crashes.

Q27. START TRANSACTION;

INSERT INTO orders

VALUES (1011, 102, CURDATE(), 'Pending', 1598.00);

INSERT INTO order_items

VALUES (5016, 1011, 206, 1, 1299.00, 0);

INSERT INTO order_items

VALUES (5017, 1011, 208, 1, 599.00, 0);

UPDATE products

SET stock_qty = stock_qty - 1

WHERE product_id = 206;

UPDATE products

SET stock_qty = stock_qty - 1

WHERE product_id = 208;

COMMIT;

	50	10:12:14	COMMIT	0 row(s) affected	0.000 sec
--	----	----------	--------	-------------------	-----------